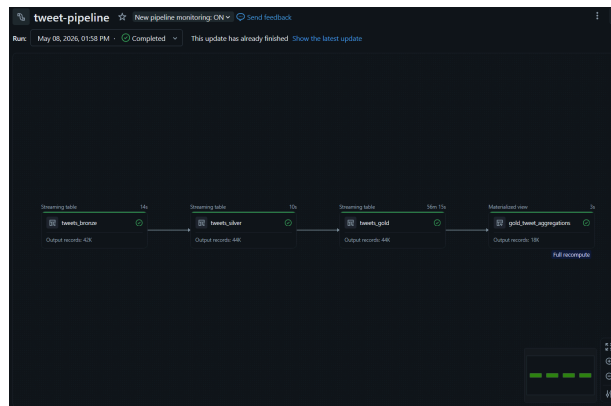


Graham Hairston
DSCC 402
4/8/2026

Tweet Pipeline Performance Analysis

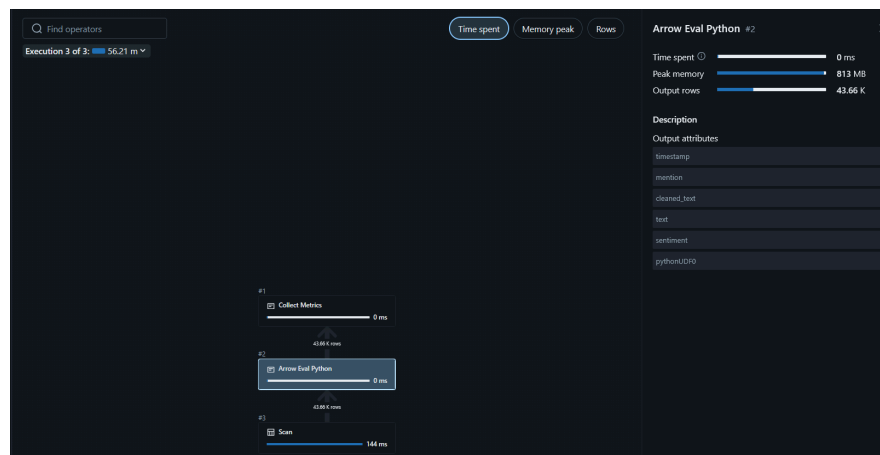
UDF Serialization Overhead

On a fresh run of the pipeline, rebuilding all tables from scratch, the slowest stage by far is the gold layer, taking close to an hour in comparison to the less than a minute of every other layer. This is most likely because of the costly UDF used to label and score the text of each tweet.



This is a difficult problem to solve completely. Ideally it would be possible to train a comparable model to the prebuilt one we have using Spark ML, as a Spark ML transformer could be properly optimized, though obviously this is not feasible in this case. Predicting in batches using `predict_batch_udf` or other similar approaches could reduce overhead to some degree.

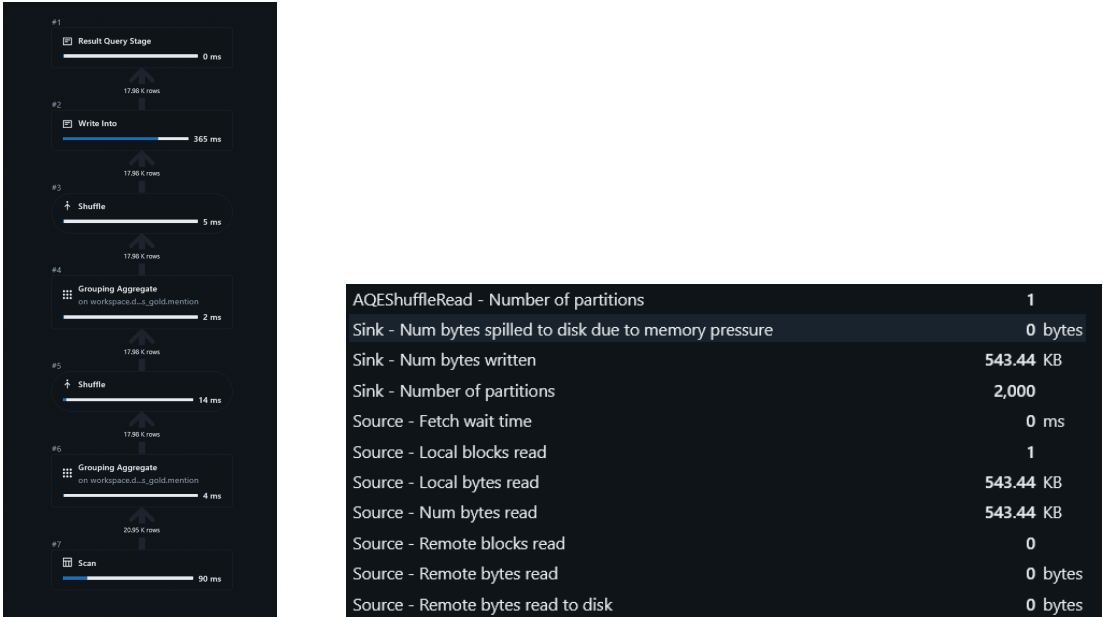
Skew



As a result of the slowness of the UDF, there is heavy skew in the gold layer, as one executor is going through all 44k rows to apply the model UDF while the rest wait. The Query

Profile does not properly capture the time spent on the UDF stage, but we can see from what it does show that other stages take very little time at all, while the conspicuously missing 65 minutes of task time is most likely entirely due to the UDF. We can also see that the step in which we produce UDF output uses quite a bit of memory compared to the single digit MB peak usage of the other operations. Again, this is a difficult problem to solve without having an equivalent Spark ML transformer model to replace our current model.

Shuffle



The aggregation layer appears to cause 2 shuffles, each with similar read/write. These shuffles are caused by grouping by “mention” as well as ordering by “total”. We could prevent one of these shuffles by not ordering by “total” in this layer. This is justified for several reasons: firstly, materialized views do not appear to support order by, so despite apparently using resources to sort the table, the output from the view is not actually ordered. Second, since the materialized view is primarily used for dashboard queries, the global sort can be removed from the view definition and applied only in the dashboard queries for which it is necessary. This reduces unnecessary shuffle work during refresh.